

Milemate

기획자를 위한 기술 기반 기획서 작성 보조 에이전트 — 설계와 검증

김재희 고재용 김선민 박하연

PART 1.

시스템 개요 - 중간발표 요약

Milemate의 4단계 / Human-in-the-loop 파이프라인

PART 2.

방법론 유효성 검증 - LLM-as-a-judge 비교 (

시나리오/평가 설계 (8차원 32항목)

결과/사례: Effort/Ablation / 차원/항목 /실제 문서 예시

PART 3.

결론

단계별 기여 / 실패 케이스 / 구조적 공백

종합 해석 / 한계 / 결론

PART 1

시스템 개요

중간발표 요약 - Milemate 파이프라인, Human-in-the-loop

Motivation

기획자의 "무엇을 만들자"와 개발자의 "어떻게 만들까" 간 간극 존재

➔ 아이디어를 구현 가능한 기술기획서로 번역하는 과정은 대부분의 조직에서 막대한 자원 소모

기획자

PLANNER'S REALITY

- 아이디어는 있지만 기술 제약 지식 부족
- 데이터 규제 관련 판단 어려움
- 기획 의도 설명 시 개발 고려 어려움
- "일단 MVP로 해보자"식 기획이 반복됨

개발자 쪽

DEVELOPER'S REALITY

- 실현가능성이 검토되지 않은 요구사항을 받음
- KPI 없이 기능부터 만들기 시작
- 개발 난항으로 인한 요구 사항 변경 및 작업 폐기 발생
- "왜 필요한지" 맥락이 사라진 스펙을 받음

아이디어를 기술 언어로 구체화하는 과정이 체계 없이 개인 경험에만 의존하는 것이 병목지점

제안 방법론

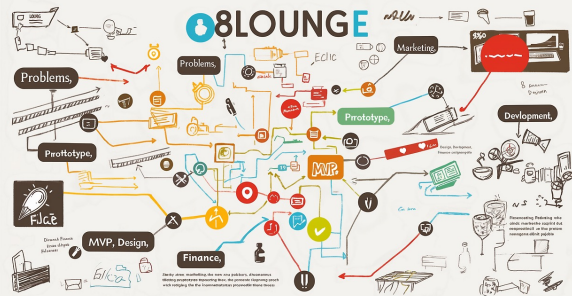
Milemate: 기획자의 자연어 아이디어를, 개발자가 바로 착수할 수 있는 기술기획서로 번역하는

기술기획 보조 에이전트 시스템

01

구조화

아이디어에 필요한
기술 의존성 정리 및 구현 가능 단위 분리



02

범위 확정

조직의 자원/데이터/규제 기반
MVP 범위 제안
내부 구현 여부 의사결정 가능

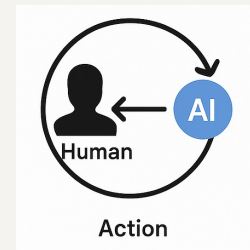
OpenAI
ChatGPT API



03

검증과 기록

Stage 기반 human-in-the-loop
워크플로우로 모든 의사결정 기록 및 검증



다양한 분야의 기획 단계에서 활용할 수 있는 기술 혁신 보조 방법론 제안

에이전트 파이프라인

기술 스택: Microsoft Agent Framework / FastAPI / vLLM / Memorization



PART 2

방법론 유효성 검증

단계형 파이프라인은 정말 더 나은 기획서를 만드는가? — 동일 입력 Ours vs Simple 비교

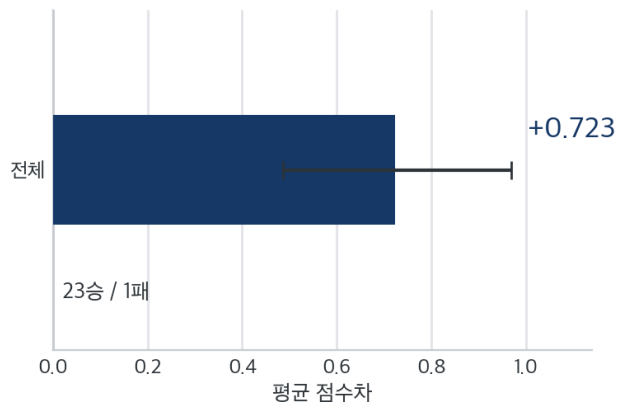
연구 질문 제안 방법론과 단순 프롬프팅 간의 비교를 통한 파이프라인 필요성 입증

RQ1. 단계형 파이프라인(Ours)이 일회성 생성(Simple)보다 더 나은 기획서를 만드는가?

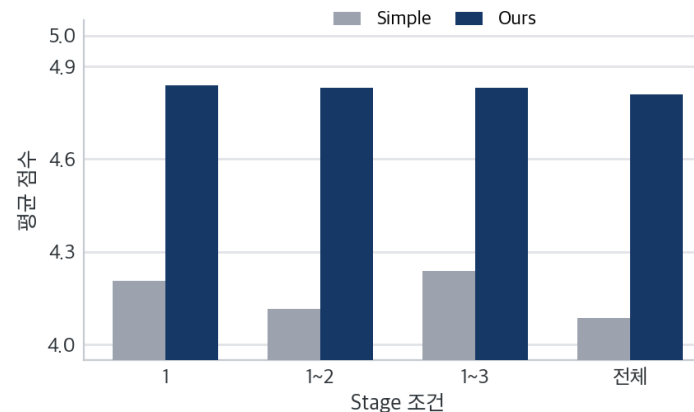
RQ2. 품질 개선이 발생하는 단계 및 속성은 무엇인가?

RQ3. 모델 성능이 실제 품질 개선에 큰 영향을 미치는가?

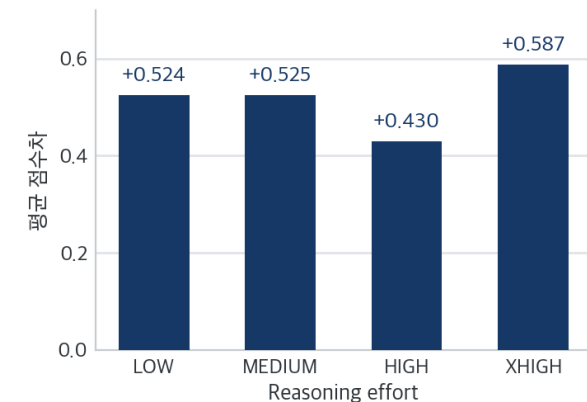
성능 개선폭



단계 별 성능 비교



모델 역량 별 성능 비교



실험 설계 - 동일 입력에 대한 생성 파이프라인 변화

동일 초기 기획 의도 / 동일 생성 모델 / 동일 토큰 예산

라스트 마일 분야 8개의 구체적 시나리오 생성 및 활용



Simple / 일회성 생성

ONE-SHOT (baseline)

중간 단계 없이 한 번에 최종 기획서 생성
일반적인 LLM 사용 방식

Ours / 단계형 파이프라인

STAGED PIPELINE (Milemate)

문제정의 → MVP범위 → 리스크검증 → 최종
각 단계 승인 / rollback 후 종합 합성

8개 라스트마일 시나리오

단일 사례가 아니라 현업에서 발생 가능한 성격이 다른 8개 운영 문제를 직접 설계
신규 도입/기존 개선/규제 민감을 의도적으로 포괄.

시나리오 목록

- ✓ 동적 배차/경로 재추천 - 피크 배차 집중 완화
- ✓ ETA 예측/지연 알림 - 지연 주문 선제 감지/안내
- ✓ 실패 배송 리스크/사전 개입 - 실패 전 미리 조치
- ✓ 신규 라이더 온보딩/이탈 방지 - 첫 2주 선제 연락
- ✓ 반품/교환 회수 예약/추적 - 역배송 흐름 신설
- ✓ 결제 전 배송비 투명 안내 - 장바구니 이탈 감소
- ✓ 매장 준비시간 지연 가시화 - 기사 헛대기 감소
- ✓ 반복 배송 문의 자동 분류 - 단순 문의 1차 자동

예시 시나리오 (동적 배차/경로 재추천)

예: 동적 배차 / 강남/서초 저녁 피크

“저녁 6~8시 강남/서초에 주문이 몰리면 잘하는 기사 몇 분께만 콜이 쏠려, 늦은 주문 하나에 사과 쿠폰 3천 원씩 나갑니다. 지금은 운영팀이 주문/기사 위치 화면을 눈으로 번갈아 보며 손으로 바꾸는데 피크엔 손이 안 따라가요.

위험한 주문이 먼저 뜨고 왜 위험한지 이유까지 보여 승인만 누르면 좋겠는데,,, 개발은 1도 몰라서 뭐부터 말해야 할지조차 모르겠어요.”

평가 척도 - LLM-as-a-judge

두 문서를 익명(문서1/2)으로 제시, 항목마다 pass(1.0)/partial(0.5)/fail(0)

8개 차원으로 구분하여 각 항목 별 성능 평가 진행

판정 방식

HOW

32개 원자 항목을 기계적으로 채점
pass/partial은 반드시 원문 인용
차원 = $1 + 4 \times (\text{획득} / \text{유효항목})$

8개 차원

WHAT

D1 문제정의 / D2 범위규율 / D3 실행구체성 / D4 측정가능성
D5 리스크/보류 / D6 내적일관 / D7 인계준비 / D8 입력충실

무엇을 보는가 — 8개 평가 차원

각 차원은 "좋은 기획서가 갖춰야 할 내용 속성"이며, 아래 32개 원자 항목으로 채점

D1 문제 정의 품질

기획자 체감 문제 구체성 / 임팩트 수치화 / 현행 법규 문제 우회 / 성공 기준 검증 가능성

D2 범위 규율

포함 범위 구체성 / 제외 범위 명시 / 자동화-사람 경계 / 범위 결정 근거

D3 실행 구체성

프론트-백엔드 정의 / 예외 상태 처리 / 운영정책 실행가능성 /
데이터 요건 특정 / 측정용 로그/이벤트

D4 측정 가능성

KPI baseline/target / 측정 방법 명시 / 측정 담당 주체 / 문제-KPI 정합성

D5 리스크/보류

시나리오 고유 실패 경로 / 보류 및 중단 발동 조건 / 과잉 자동화 가드레일 /
개인정보 및 민감정보 취급

D6 내적 일관성

문제→범위→지표 흐름 단절 / 문서 내 모순 / 정책-설계 충돌

D7 의사결정/인계

결정 안건 완결성 / 미해결 질문 추적 / 개발팀 인계 질문 및 요청 / 결재 판단 완결성

D8 입력 충실/간결

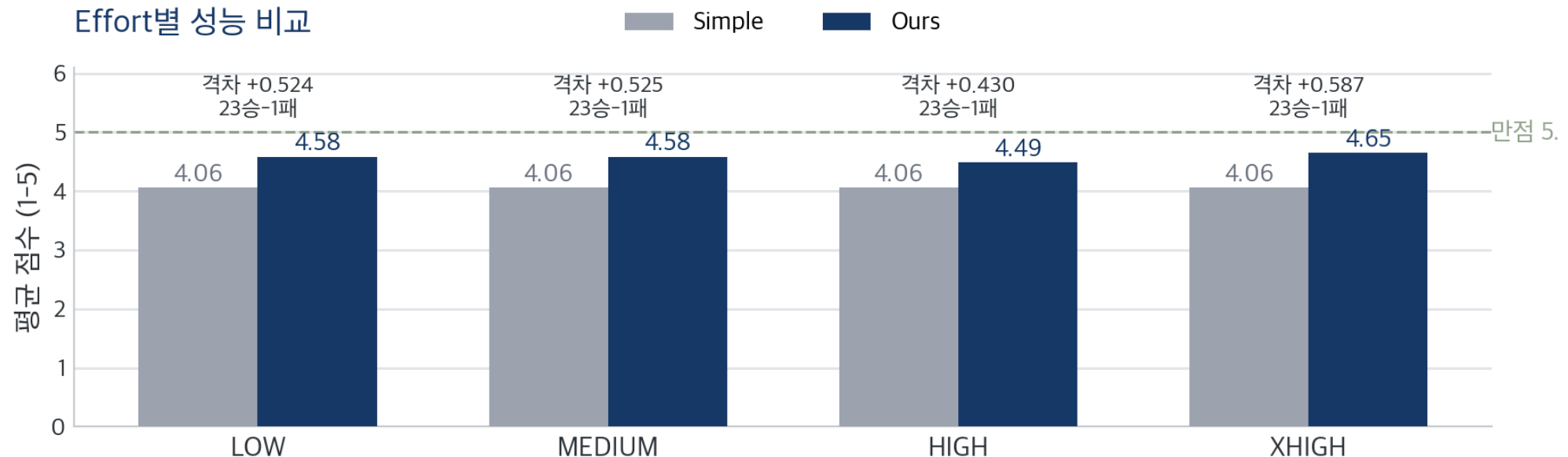
화자 우려 및 제약 반영 / 수치 출처 추적 / 요청 방향 유지 / 효과 주장 간결성

Reasoning 예산 별 성능 비교

Agent의 경우 reasoning 예산을 증가시킬 경우 일반적으로 성능 개선 발생

reasoning effort를 low→xhigh로 올려도 격차는 유지되고, 절대 점수도 Ours가 만점에 근접.

➔ 단순한 프롬프팅으로 적절한 제안서 작성의 어려움 존재

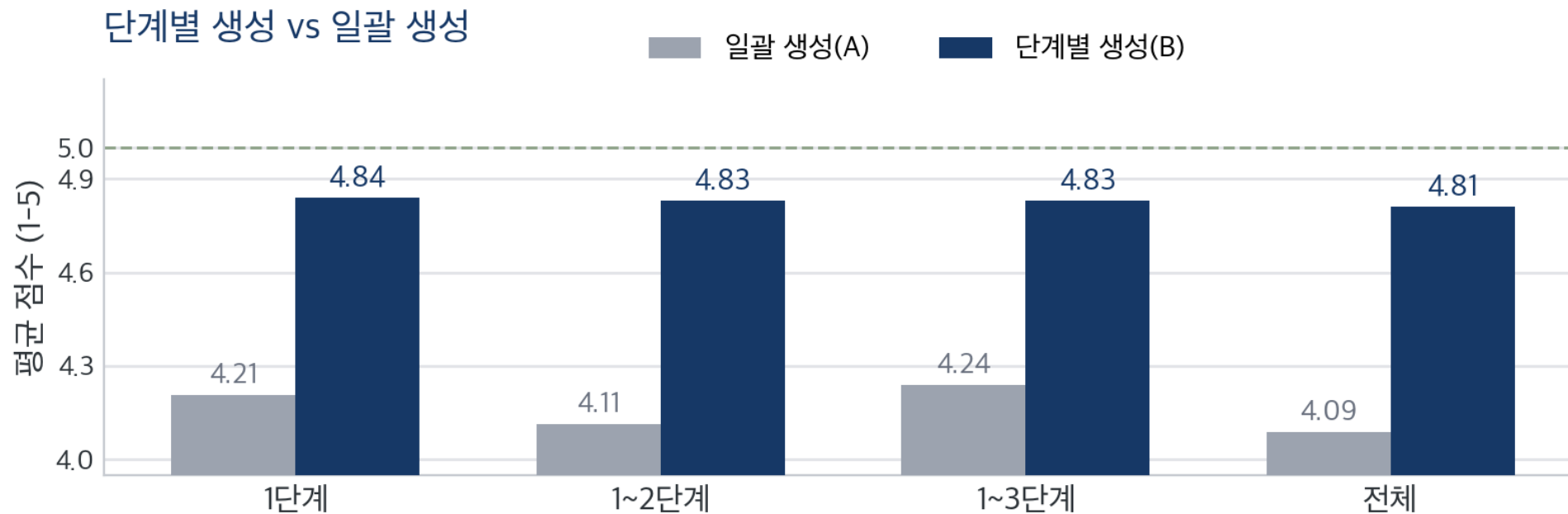


Stage 별 성능 비교

제안 방법론의 각 단계별 필요 요소 만족 확인

제안 방법론의 일부 요소 제거 시 성능 저하 확인

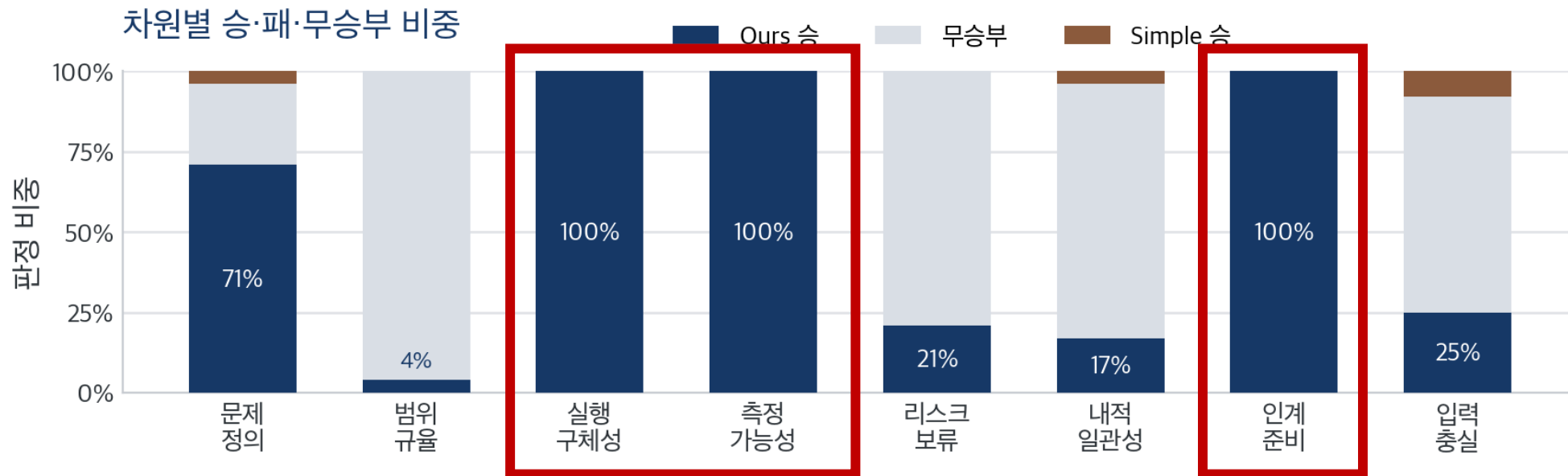
➔ 제안 방법론의 모든 요소 필요성 확인



평가 차원별 상대 비교

차원별 각 기획 결과물에 대한 1대 1 비교 결과 대부분의 경우 제안 방법론 우위 or 동등

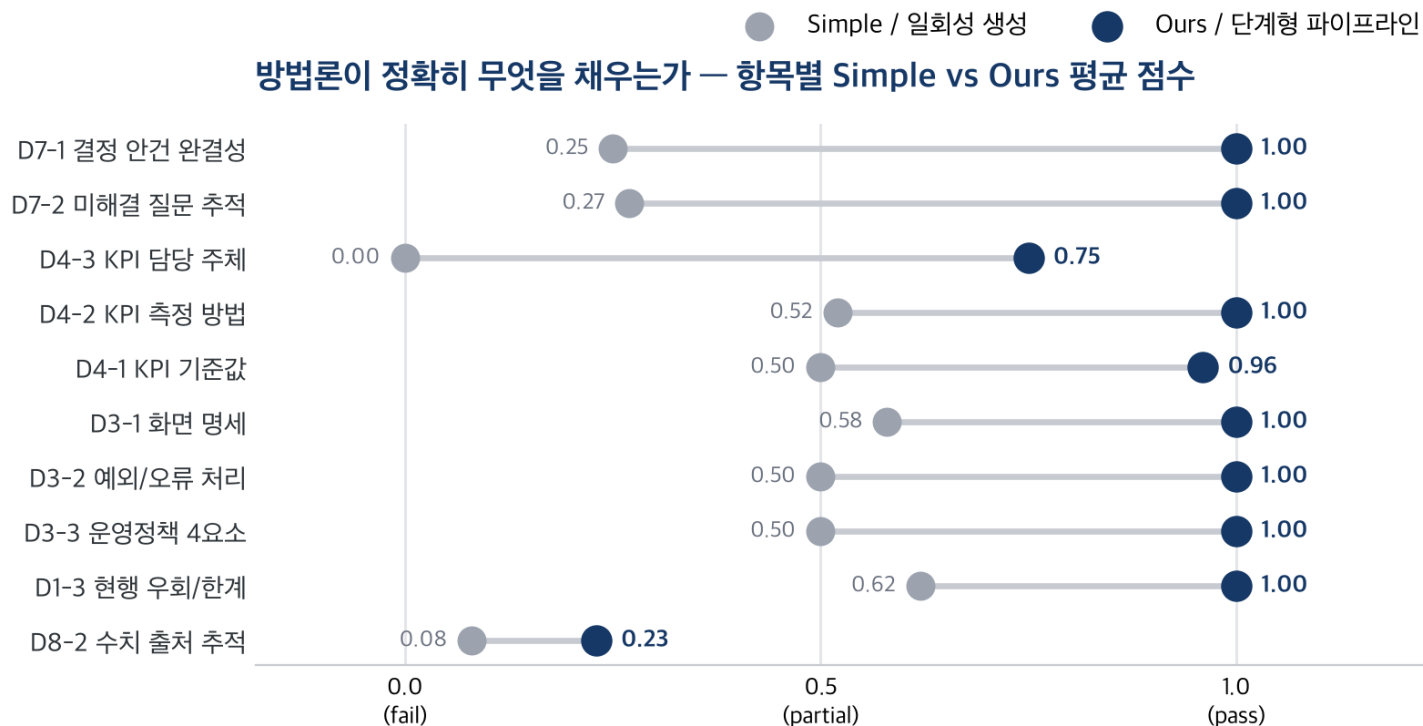
➔ 실행 구체성, 측정 가능성, 인계 준비 등의 개발자 중심 요소에 대해 매우 높은 성능 확인



방법론 개선 중심 요소

단순 프롬프팅 시 개발 관련 요소에 대해 충분한 생성 X

➔ 기획 및 개발 시의 협업 구조를 고려한 파이프라인을 통하여 점수 개선 확인



13. 단계별 핵심 요약

실제 생성 예시

사용자 초기 입력: “저녁 6~8시 강남/서초에 주문이 몰리면 잘하는 기사 몇 명한테만 **콜이 쏠려요**. 늦은 주문 하나에 사과 쿠폰 3천 원씩 날아가고요. 시스템이 기사를 확 바꾸면 반발할까 무섭고... **위험한 주문**이 ‘왜 위험한지’ 이유까지 떠서 **운영자가 승인**만 누르면 좋겠는데, 개발은 1도 몰라서 뭐부터 말해야 할지 모르겠어요.”

단계	판단 근거	기획자 제공 정보	개발자 제공 정보	PRD 핵심 정보
1단계 문제 정의	‘자동 배차’가 아니라 위험 주문 우선순위화 + 운영자 승인 으로 문제를 재정의. 추천 근거 표시·승인 이력·KPI 계측을 1단계 범위로 설정.	픽크 때 봐야 할 위험 주문만 노출 , 추천 사유와 기사 부담 기준을 확인한 뒤 승인하는 흐름.	위험점수 / 추천후보 / 승인 워크플로API 분리. 추천 생성과 배차 실행 사이에 ‘승인’ 상태 필수 .	위험·보상·과부하 큰 주문만 대시보드상단에 올리고, 사유 확인 후 승인하는 승인형 배차 추천 MVP 정의.
2단계 서비스 구조	완전 자동 배차를 제외하고 승인·보류·거절 흐름 확정. 지연률 / SLA / 배송원 편차 / 보상 비용을 같은 화면·KPI체계로 통합.	확인 → 사유 이해 → 승인/보류/거절 → 파일럿 KPI 리뷰까지의 운영 화면 정의. 현장 반발을 줄이는 의사결정 흐름이 핵심.	추천 상태 모델 (generated/approved/held/rejected...)정의. approved 상태만 실제 배차 변경 요청 을 생성.	1단계 대시보드 범위를 유지하며 승인·보류·거절 기록과 4대 지표 리뷰 흐름 을 확정.
3단계 검증 / 리스크	데이터가 늦거나 공급 부족이 구조적이면 추천을 실행하지 말고 보류·수동확인·고객안내로 전환 해야 한다는 게이트 기준 추가.	자동화 강화가 아니라 ‘추천을 믿어도되는 조건’ 을 정의. 오래된 데이터·공급 부족 시 승인보다 보류·롤백 권고.	생성·승인 직전 데이터 신선도 재검증게이트 . 실패는 blocked_for_stale_data 로 기록, 화면 배지=API 신뢰도 일치.	데이터 지연 / 공급 부족 / 수요 초과상황에서 믿을 추천 vs 보류할 판단 의기준을 정의.
4단계 최종 기획서	조기 발견 + 납득 가능한 추천으로 고객 신뢰·기사 부담·매장 클레임·보상비용 을 함께 절감하는 최종 서비스 시나리오로 정리.	문제 재정의 — 기사 자동 교체가 아니라 늦어질 주문을 먼저 찾아 운영자가근거로 승인 하게 만드는 것.	필수 데이터 묶음(주문/위치/ETA/공급/매장/안내/비용/보존정책) + 기술블록·제약· 구현 순서·검증 계획 확정.	3단계 신뢰도·보류 기준 위에 설명 가능한 추천을 승인 해 지연 비용·신뢰 리스크를 줄이는 최종 PRD 확정.

12. 예시로 보는 차이 ①

바람직한 조건을 누가 채우나

사용자 질문 (동적 배차)

“위험한 주문이 먼저 뜨고, 왜 위험한지 이유까지 보여 운영자가 승인만 누르면 좋겠어요. 그런데 개발은 1도 몰라서 뭐부터 말해야 할지 모르겠어요.”

1. 결재 요약

결론: 조건부 진행 권고. 이번 MVP는 자동 배차 시스템이 아니라, 강남-서초 18:00~20:00 피크타임에 지연 위험 주문을 먼저 띄우고, 운영자가 추천 사유를 확인한 뒤 승인 또는 보류 하는 운영 보조 콘솔이다.

핵심 메시지는 명확하다. 피크타임에 밀리는 주문을 먼저 발견하고, 운영자가 납득할 수 있는 추천으로 배송 지연과 보상 비용을 줄인다. 고객에게는 늦기 전에 안내하고, 배송원에게는 특정 기사 쏠림을 줄이며, 매장에는 책임 전가가 아닌 현장 상태 기반 조정으로 설명한다.

외부 근거상 온디맨드 배송은 수요가 무작위로 몰리는 환경에서 동적 배차-라우팅이 성과를 낼 수 있는 영역이다. 실제 데이터 기반 연구에서도 동적 배차 프레임워크가 기존 정책 대비 평균 배송시간을 개선한 사례가 있다. 다만 사람 승인형 시스템은 “사람을 루프에 넣는 것”만으로 충분하지 않고, 운영자가 판단할 수 있는 맞춤형 설명과 역할 정의가 필요하다. (arxiv.org) (arxiv.org)

2. 문제 정의와 MVP 목표

구분	현재 문제	MVP 해결 방향
지연 주문	피크 때 주문 화면과 기사 위치 화면을 운영자가 수동 비교	SLA 초과 가능성이 큰 주문을 위험도순으로 자동 정렬
배송원 편차	잘하는 기사에게 콜이 몰리고 일부 기사는 비어 있음	배송원별 활성 주문 수, 예상 작업시간, 과부하 지표를 함께 표시
CS/보상 비용	늦어진 뒤 CS가 들어오고 건당 3,000원 쿠폰 비용 발생	지연 예상 건수와 보상비 노출액을 사전에 계산
운영 신뢰	추천 이유가 없으면 운영자가 화면을 믿지 않음	“왜 위험한지”와 “왜 이 기사/조치를 추천하는지”를 카드로 설명

13. 예시로 보는 차이 ②

Ours만 만들어내는 운영 구조

Simple엔 없던 운영 구조가 단계에서 생성된다. 각 사례의 질문 / 바람직한 조건 / Ours가 만든 구조.

반품 회수 예약

Stage 3 / 타당성 검증

질문

“물건을 언제 가지러 오는지 정해진 게 없어요”

바람직한 조건

초과 예약 없이 안전하게 자리를 잡아야 한다



실패 배송 개입

Stage 3 / 타당성 검증

질문

“실패가 터진 다음에야 수습을 시작해요”

바람직한 조건

여러 신호로 개입 수준을 미리 정해야 한다



동적 배차 인계

Stage 4 / 최종화

질문

“개발팀에 뭐부터 말해야 할지 모르겠어요”

바람직한 조건

개발이 바로 연동할 인터페이스가 있어야 한다



PART 3

결론

Milemate 기여점 및 한계점 정리

실제 단계별 추가 정보

사용자 초기 입력: “저녁 6~8시 강남/서초에 주문이 몰리면 잘하는 기사 몇 명한테만 **콜이 쏠려요**. 늦은 주문 하나에 사과 쿠폰 3천 원씩 날아가고요. 시스템이 기사를 확 바꾸면 반발할까 무섭고... **위험한 주문**이 ‘왜 위험한지’ 이유까지 떠서 **운영자가 승인**만 누르면 좋겠는데, 개발은 1도 몰라서 뭐부터 말해야 할지 모르겠어요.”

단계	PRD 핵심 정보	
1단계 문제 정의	위험·보상·과부하 큰 주문만 대시보드 상단에 올리고, 사유 확인 후 승인하는 승인형 배차 추천 MVP 정의.	✓ KPI 구조화 ✓ 문제 해결을 위한 서비스 초안 기획
2단계 서비스 구조	1단계 대시보드 범위를 유지하며 승인·보류·거절 기록과 4대 지표 리뷰 흐름을 확정 .	✓ 구체적 서비스 시나리오 구성 ✓ 서비스 내 심화 요소 설정
3단계 검증 / 리스크	데이터 지연 / 공급 부족 / 수요 초과상황에서 민을 추천 vs 보류할 판단 의 기준을 정의.	✓ 문제 상황 설정 ✓ 각 상황 별 기준 설정 ✓ 해결 방안 제시
4단계 최종 기획서	3단계 신뢰도·보류 기준 위에 설명 가능한 추천을 승인 해 지연 비용·신뢰 리스크를 줄이는 최종 PRD 확정.	✓ 전체 기획서 작성 ✓ 개발자 및 기획자 모두 이해 가능한 포괄적 정보 포함 ✓ 구체적 개발 지시서 초안 작성

기여점

- ✓ Human-in-the-Loop 구조를 통한 신뢰도 높은 기술혁신 도구 개발
 - 기획자 요청 → 모델 생성 및 결과물 정리 → 기획자 승인 및 피드백 구조
- ✓ 기획 및 개발 간 의사소통 개선을 통한 기술혁신 병목 요소 해소
 - 각 단계별 기획자 및 개발자 용 문서 작성, 최종 개발을 위한 PRD 초안 제공
- ✓ 서비스 혁신 프로세스 상 품질 측정 요소 제시
 - LLM-as-a-Judge 기반 8개 차원 32개 요소 품질 측정 구체화

한계점

- ✓ 실제 검증이 진행되지 않음
 - 현실문제를 통한 기획자, 개발자 사용 검증 미비
- ✓ 업무 친화적이지 않은 보고서 형식
 - 단순 PDF 파일 제공으로 인한 후속 보고서 작성 작업 필요

감사합니다.